Ilja Safronovs

R&D engineer, 3D reconstruction. Giraffe360, Riga, Latvia.

ilya.safronos@gmail.com | github.com/ilyaeis | linkedin.com/in/i%C4%BCja-safronovs-eis | ilyaeis.github.io

I work on 3D Gaussian splatting at Giraffe360: a pipeline that turns sparse 360° panoramas and LiDAR scans into splats, in PyTorch and CUDA. Most of my work starts with a reconstruction that looks wrong. **I find the cause and fix it in our training code.** I pick each fix in controlled training runs against the old default. Two papers went from PDF to my code: Taming 3DGS in PyTorch and Analytic-Splatting in CUDA. I mentored a junior engineer and led three engineers on a new 3D reconstruction feature. **Next, I want to grow into leading a team.**

Experience

Giraffe360, R&D engineer — Riga, Jul 2025 to present

Giraffe360 makes cameras and software for real-estate virtual tours. I am in the R&D team, on the 3D reconstruction pipeline: camera poses from structure from motion and bundle adjustment (COLMAP), Gaussian splat training with gsplat, and the camera paths of the videos.

- **Blurry windows, outdoor views and mirrors in our 3D videos.** I found the cause and tested fixes in training runs until the haze and floaters were gone. Builds on the team's earlier window masks. **Default in all Giraffe360 Gaussian splat videos since June 2026.** Before and after: instagram.com/p/DZsNRB8o772
- **Our training defaults use my Gaussian pruning and relocation rules** (Jan to Jul 2026), inside an MCMC training strategy. They build on Taming 3DGS (Mallick et al., 2024), which I had implemented from the paper, and on the team's influence tracking: a Gaussian that adds little to the renders gets pruned or relocated. Together, the Jul 2026 default set (my relocation rule with a new optimizer and a depth mask) trains 13 to 16 % fewer Gaussians at equal or better quality.
- **Implemented Analytic-Splatting (Liang et al., 2024) in gsplat**: CUDA forward and backward kernels, written from the paper. We use it for downsampling.
- Made training faster: 8.6 % on an RTX 3090 at the same quality, by dropping a per-step opacity cull; on one large scene a step went from 117.6 to 68.1 ms (42 % less) with PSNR unchanged.
- **Mentored a junior engineer on a new depth method** (denser LiDAR depth). Mostly his work. Our splat training has used it as its depth prior since October 2026.
- **Led three engineers on a new 3D reconstruction feature** (Oct 2026). Most of the code is mine, from the camera poses (SfM and bundle adjustment) to the training recipe; I reviewed and merged theirs and cut the release. In production. I have cut the releases of our 3D pipeline since September 2026.
- **Looped drone video**, one flight per room. My idea from the Feb 2026 hackathon (I submitted 6 ideas; 3 of them were among the 51 recognised). I wrote the camera path and reworked the video pipeline. The look was worked out with the designers. **It is a product now**: instagram.com/giraffe360/reel/Dd9ODBVIHhE
- Fixed the camera height in rooms on a half floor, for all video types. A long-standing bug.
- **Team tools.** An experiment runner: an AI agent starts and tracks trainings on a local GPU. An MCP server (Node.js) for our task board. Ground-truth datasets and checks for splat quality. I also led the move of our 3DGS code into one monorepo, with a code-size check (2026).
- **A Rust/wgpu splat viewer that other apps can embed and control** (May 2026). It swaps and combines PLY scenes live, and shows point clouds. Before that, a side-by-side comparison tool with camera sync.
- **Prototypes**: mirror reflections done right, meshes from splats, a stage that cuts the splat count for viewers, a fly-in video intro. Got Difix and NVIDIA's fixer running on our GPUs for the team to try (Feb 2026).
- Jul to Oct 2025: a small service that wrote tour narration with the OpenAI API and AWS S3.

Education

Riga Technical University, Professional Bachelor, Financial Engineering — 2024 to 2028 (expected)

Probability, statistics, stochastic analysis, Monte Carlo methods, algorithms, databases.

AI Talent Program (Ponder) — Nov 2024 to Jan 2025

C++, Python, maths and AI, with mentors from industry. That is how I got the Giraffe360 job.

Riga State Classical Gymnasium, programming and mathematics track

State mathematics olympiads. Small Mathematics University (University of Latvia).

Projects

- **Portfolio site** (2026): ilyaeis.github.io. A live Gaussian splat of a flat that I trained with gsplat on public Zip-NeRF photos, 18 minutes on one RTX 3090.
- **sort_visualization** (2022, C++/SFML): 16 sorting algorithms animated as columns, points, circles, spirals and pyramids. The same C++ runs in the browser, compiled to WebAssembly.
- **Coding-agent harness** (own time): one master session plans and checks the work; workers run in their own git worktrees.
- **monte-carlo-finance** (2026, Python): option pricing with variance reduction, GARCH volatility, VaR backtests with the Kupiec test, efficient frontier, CAPM. From RTU coursework.
- **projects_with_NEAT** (2025, Python): NEAT agents that learn Flappy Bird and Pong.
- **Offline Graphing Calculator** (2023, with Iļja Volinskis): 4th of 27 entries in the Riga Business School programming competition for school students.
- Also: load-balancing-experiments (2026, Python, RTU networks course: DNS, queueing, consistent hashing), crypto-quant-lab (2026, Python, trading-rule backtests), flow (2026, Dart/Flutter, private: offline workout tracker with golden-image tests in CI), RTU_program_parser (2025, Python, with Aleksejs Peņkovs: finds extra RTU lectures that fit your schedule).

Skills

3D: Gaussian splatting with gsplat (MCMC, pruning and relocation, custom CUDA kernels), SfM and bundle adjustment with COLMAP, equirectangular and fisheye camera models, LiDAR depth priors.

Python, C++, CUDA, PyTorch, gsplat, Rust/wgpu, Dart/Flutter, JavaScript/three.js, Java, SQL.

Linux, Docker, Git and GitLab CI, AWS S3, Node.js (MCP servers), OpenAI API.

Latvian, English, Russian.

With the R&D team at Giraffe360. Methods build on Taming 3DGS (Mallick et al., 2024) and Analytic-Splatting (Liang et al., 2024).